

Rediff Chat: Enterprise Database Architecture

System Component: Core DB & ejabberd Storage Engine Integration

1. Overview

Rediff Chat is a high-performance enterprise messaging platform engineered over the **ejabberd** XMPP ecosystem. To ensure operational isolation, compliance, and enterprise scalability, the platform segregates state across two distinct logical database engines:

Database A: Rediff Application Database

Acts as the centralized system of record for corporate operations and identity control.

- User Corporate Identities & Directory
- Authentication Proofs & Status
- Detailed Employee Metadata
- Organizational Hierarchy & Structures
- Application-Level ACLs & Permissions

Database B: ejabberd Database

Optimized for high-throughput, low-latency transactional XMPP messaging operations.

- XMPP Protocols Accounts
- Rosters & Contact Relationships
- Real-time Presence Information
- Active Cluster Sessions & State
- Message Routing Matrix
- Offline Message Spooling
- Multi-User Chat (MUC) Rooms

2. System Architecture Layout

The topological orchestration outlines how client state is synchronized. The Rediff App handles upstream user state lifecycle events and translates them downward to provision the underlying XMPP cluster layers.



3. Rediff Database Schema Definitions

The following structures define the structured schemas residing inside the primary enterprise relational repository.

Table: users

Main directory tracking active platform resources.

```
CREATE TABLE users (  
  id          BIGINT PRIMARY KEY,  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  username    VARCHAR(100) UNIQUE NOT NULL,  
  full_name   VARCHAR(255),  
  status      VARCHAR(20),  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

Data Sample:

ID	USERNAME	EMAIL	STATUS
101	alice	alice@rediff.com	ACTIVE
102	bob	bob@rediff.com	ACTIVE

Table: user_auth

Handles cryptographic verification metadata.

```
CREATE TABLE user_auth (  
  user_id      BIGINT PRIMARY KEY FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE  
  password_hash VARCHAR(255) NOT NULL,  
  last_login_at TIMESTAMP  
);
```

Table: user_profile

Captures dynamic meta-information mapped to corporate identity.

```
CREATE TABLE user_profile (  
  user_id      BIGINT PRIMARY KEY FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE,  
  designation VARCHAR(100),  
  phone        VARCHAR(50),  
  avatar_url   VARCHAR(500)  
);
```

4. ejabberd Native Database Schema

These logical tables map closely to Erlang Mnesia tables or integrated relational data backing the XMPP cluster nodes.

Table: passwd

Stores local XMPP account routing authentications. Native Erlang structural pattern: `[us, password]`.

```
CREATE TABLE passwd (  
    jid          VARCHAR(255) PRIMARY KEY,  
    password_hash VARCHAR(255) NOT NULL  
);
```

Logical Sample:

JID (PRIMARY KEY)

alice@chat.rediff.com

bob@chat.rediff.com

Table: roster

Maintains peer-to-peer connection topologies. Native structure: `[usj,us,jid,name,subscription,approved,ask,groups,...]`.

```
CREATE TABLE roster (  
    owner_jid    VARCHAR(255),  
    contact_jid  VARCHAR(255),  
    display_name VARCHAR(255),  
    subscription VARCHAR(50),  
    groups       TEXT,  
    PRIMARY KEY (owner_jid, contact_jid)  
);
```

Data Sample:

OWNER_JID	CONTACT_JID	DISPLAY_NAME	SUBSCRIPTION
alice@chat.rediff.com	bob@chat.rediff.com	Bob Roberts	both

Table: session

Maintains persistent active state clusters for connected entities.

```
CREATE TABLE session (
  session_id VARCHAR(255) PRIMARY KEY,
  jid        VARCHAR(255) NOT NULL,
  resource   VARCHAR(100),
  login_time TIMESTAMP
);
```

Table: offline_msg

Transient pipeline queue storing messages deferred for offline recipients.

```
CREATE TABLE offline_msg (
  message_id    BIGINT PRIMARY KEY,
  recipient_jid VARCHAR(255) NOT NULL,
  sender_jid    VARCHAR(255) NOT NULL,
  message_body  TEXT,
  created_at    TIMESTAMP
);
```

Table: muc_room

Persistent storage for corporate Group Chat environments.

```
CREATE TABLE muc_room (
  room_jid  VARCHAR(255) PRIMARY KEY,
  room_name VARCHAR(255),
  created_by VARCHAR(255),
  created_at TIMESTAMP
);
```

Table: muc_online_room & route

Used natively for session node telemetry mapping and message execution pipelines.

```
-- Tracking volatile room context
CREATE TABLE muc_online_room (
  room_jid  VARCHAR(255) PRIMARY KEY,
  node      VARCHAR(100),
  active_users INT
);

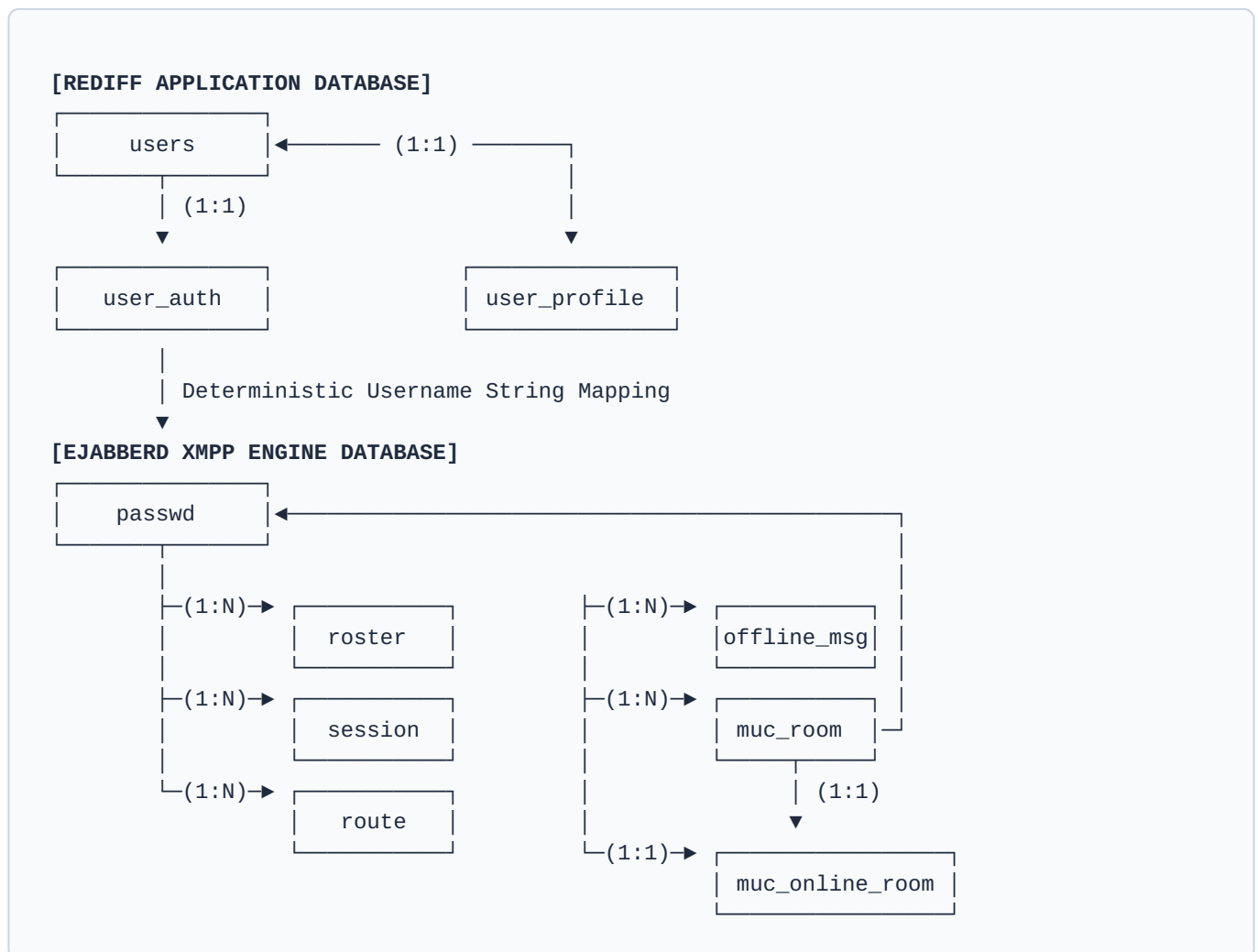
-- Core engine message delivery matrix
CREATE TABLE route (
  jid        VARCHAR(255),
  node       VARCHAR(100),
  resource    VARCHAR(100),
  PRIMARY KEY(jid, node, resource)
);
```

5. Identity Cross-System Mapping & Entity Relationships

The mapping configuration enforces a rigid deterministic design. There are no relational foreign key loops across the databases; instead, identifiers resolve explicitly via tokenized formatting convention mappings:

Rediff User ID Reference → **users.username (e.g., alice)** → **XMPP Identifier:**
username@chat.rediff.com

Structural Entity Relationship Topology



6. Automated Account Lifecycle Flows

To avoid race conditions or broken connections, orchestrators must handle system updates in transactional steps.

Flow A: Account Creation Pipeline

STEP 1 Upstream Transaction Execution

Write standard operational user attributes to the core database infrastructure.

```
INSERT INTO users (id, email, username, full_name, status)
VALUES (101, 'alice@rediff.com', 'alice', 'Alice Smith', 'ACTIVE');
```

STEP 2 XMPP Engine Account Sync

Invoke the ejabberd engine CLI API daemon to establish isolated routing boundaries.

```
ejabberdctl register alice chat.rediff.com structural_secure_password_hash
```

Flow B: Account Deletion Pipeline

STEP 1 Upstream User Purge

Execute cascade-compliant drop routines inside the primary relational tables.

```
DELETE FROM users WHERE id = 101;
```

STEP 2 Engine De-provisioning Command

Deregister standard routing matrices from the runtime messaging cluster layers.

```
ejabberdctl unregister alice chat.rediff.com
```

7. Data Ownership Architecture Matrix

DOMAIN CATEGORY	SYSTEM OWNER	ARCHITECTURAL OPERATIONAL PURPOSE
User Identity	Rediff DB	Core platform source of truth for global personnel records.
Password / Auth	Rediff DB	Central authority handling cryptographically signed security checks.
Profile Data	Rediff DB	Non-realtime business metadata (designations, phone numbers, links).
Employee Data	Rediff DB	Company directory structures and internal reporting chains.
Contact List / Roster	ejabberd DB	XMPP buddy list structures tracking direct operational peers.
Presence State	ejabberd DB	Highly volatile memory storage capturing online status and updates.
Session Routing	ejabberd DB	Network maps allocating data packets across functional cluster nodes.
Offline Messages	ejabberd DB	Transient standard disk spool tracking unread message packets.
Chat Rooms (MUC)	ejabberd DB	Multi-user chat structural contexts, policies, and permissions.

8. Recommended Enterprise Model Strategy

Best Practice: ejabberd External Authentication Implementation

To minimize sync code loops, race conditions, and password duplication risks, it is highly recommended to transition to an **ejabberd External Authentication script model**. Under this topology, ejabberd does not duplicate credentials locally. Instead, it hooks runtime connection handlers directly to verified external authentication daemons or API interfaces exposing the Rediff Database system of record.

Implementing this decoupling strategy guarantees strict enterprise integrity: clear system boundaries are preserved, maintenance surface area decreases significantly, and the chat layer scales independently without side effects on core enterprise data layers.